

SugarSense

Early Diabetes Risk Screening with Supervised Machine Learning

Mini Project | Module: Supervised Learning | Individual or pairs | Duration: 5-7 days

1. Background: a real-life problem

Diabetes is one of the most widespread chronic diseases in the world, and India carries one of the heaviest burdens. The danger is that many people stay undiagnosed for years: a proper lab work-up costs money and time, and a village clinic or community health worker cannot send everyone for it. Late detection means serious complications later (heart, kidney, eye and nerve damage).

Imagine you are the data scientist of a public-health NGO. The health workers collect a few **cheap, quick measurements** (age, BMI, blood pressure and similar). They need a tool that says: *"This person looks high-risk, send them for a full lab test."*

2. Problem statement

Build a supervised machine-learning model that predicts whether a patient is likely to have diabetes (Outcome = 1) from 8 clinical measurements. Compare all the algorithms we have studied, choose the best one using proper validation, and tune it for a medical goal: missing a diabetic patient is worse than raising a false alarm.

Type of task: binary classification (with one small regression bonus). **Main metric:** ROC-AUC and Recall for the diabetic class. Accuracy alone is not enough, and you will see why.

3. What you will practise

Topic from the course	Where it appears in this project
Python, NumPy, Pandas	Loading, auditing and cleaning the data, building new columns
Matplotlib / Seaborn	EDA plots, model comparison charts, ROC / PR curves, confusion matrix
Git	Keep the project in a Git repository; commit after every phase (at least 6 commits)
Linear Regression	Bonus task: estimate glucose from cheap measurements (R^2 , MAE)
Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost, SVM	Train and compare all six on the same split
Train/test split, cross-validation, overfitting	Stratified split, 5-fold CV, train-vs-validation gap
Hyper-parameter tuning, evaluation metrics	RandomizedSearchCV, precision / recall / F1 / AUC, threshold tuning

4. Dataset

Name: Pima Indians Diabetes Database (originally from the US National Institute of Diabetes and Digestive and Kidney Diseases). **Size:** 768 patients, 8 input features, 1 target. All patients are females aged 21 or above. The file `diabetes_screening_data.csv` is provided with column headers. The same data is on Kaggle ("Pima Indians Diabetes Database") and on the UCI / OpenML repositories if you

want to read about it.

Column	Meaning	Unit / note
Pregnancies	Number of times pregnant	count
Glucose	Plasma glucose after a 2-hour oral glucose tolerance test	mg/dL
BloodPressure	Diastolic blood pressure	mm Hg
SkinThickness	Triceps skin-fold thickness (body-fat indicator)	mm
Insulin	2-hour serum insulin	mu U/ml
BMI	Body mass index	kg/m ²
DiabetesPedigreeFunction	Score of diabetes history in the family	higher = more family history
Age	Age of the patient	years
Outcome	Target: 1 = diabetic, 0 = not diabetic	about 35% are 1

Detective warning. This dataset is not as clean as it looks. `df.isna().sum()` prints zeros everywhere, but `df.describe()` tells a different story. Finding what is wrong, and deciding how to fix it without leaking information, is the first test of this project.

5. Tasks (follow the phases in order)

Phase 1: Data audit

- Show shape, data types, `describe()`, duplicates and class balance.
- Find every column that contains **impossible values**. Explain in one sentence why they are impossible and how you will treat them.
- Calculate the % of problematic values in each column and show it in a bar chart.

Phase 2: EDA

- Plot the distribution of every feature separately for diabetic vs non-diabetic patients.
- Draw a correlation heatmap. Which 2-3 features look most useful? Which look weak?
- Write 4-5 lines of observations in a Markdown cell (these are marked).

Phase 3: Feature engineering

- Create **at least 3 new features** using domain logic (examples: a flag for a missing measurement, an obesity flag from BMI, a product of two risk factors). Justify each in one line.
- Only create features that use values from the same row. Anything that needs the mean / median of a column must wait until the pipeline (Phase 4).

Phase 4: Split and pipelines (avoid data leakage)

- Use a **stratified** 80/20 train-test split with `random_state=42`. Do not touch the test set until Phase 7.
- Build a Pipeline per model: imputer, then scaler (only where needed), then model. Explain in a comment why KNN / SVM / Logistic need scaling but trees do not.
- Handle the class imbalance (for example `class_weight='balanced'` or `scale_pos_weight`).

Phase 5: Compare all six models

- Train Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost and SVM using **5-fold stratified cross-validation**.
- Report accuracy, precision, recall, F1, ROC-AUC (mean and std) **and the train score** in one comparison table, plus one bar chart.
- Add an "overfit gap" column (train score minus validation score). Which models overfit? Which are stable?

Phase 6: Tune

- Tune at least **three** models with RandomizedSearchCV (scoring = roc_auc). Show before/after scores.
- Select your final model using **cross-validation scores only** (never by peeking at the test set).

Phase 7: Final test evaluation

- Evaluate the tuned models once on the test set: metrics table, ROC curves, Precision-Recall curves, confusion matrix of your best model.
- Report in words: how many diabetic patients were missed (false negatives) and how many healthy patients were wrongly flagged (false positives).

Phase 8: Think like a doctor (threshold tuning)

- The default threshold of 0.5 is arbitrary. Using **out-of-fold predictions on the training data**, pick a threshold that gives **recall of at least 85%** with the best possible precision.
- Compare default vs tuned threshold on the test set and explain the trade-off for a health worker in 3-4 lines.

Phase 9: Explain and conclude

- Show feature importance (permutation importance or tree importance) and discuss whether it matches your EDA.
- Write a conclusion: best model, what surprised you, limitations of the data, and why this tool must not replace a doctor.

6. Bonus challenges (extra marks)

- **Regression:** use Linear Regression to predict *Glucose* from the cheap measurements only. Report R^2 and MAE and say whether it could replace the lab test.
- **Learning curve:** plot score vs training-set size. Would collecting more data help?
- **Mini app:** write a function `screen_patient(...)` that returns a Low / Medium / High risk message for one new patient, and save the model with `joblib`.
- **Compare imputers:** median vs KNN imputation vs "drop rows". Which is best and why?

7. Rules

- Use only the algorithms and libraries from our course (NumPy, Pandas, Matplotlib/Seaborn, scikit-learn, XGBoost).
- Fix `random_state=42` everywhere so your results are reproducible.
- The test set is used **once**, at the end. Fitting an imputer, scaler or selecting a model on test data counts as data leakage and loses marks.

- Every plot needs a title and axis labels; every section needs a short Markdown explanation. The notebook must run top to bottom (*Restart & Run All*) without errors.
- Copying code is fine for syntax; copying conclusions is not. Be ready to explain any line in a short viva.

8. Deliverables

#	Deliverable	Format
1	Jupyter notebook with code, outputs and Markdown explanations	.ipynb
2	Git repository link (README with problem, approach, results, how to run; at least 6 meaningful commits)	GitHub URL
3	1-page summary: best model, key metrics, threshold choice, limitations	PDF or Markdown
4	(Optional) saved model + screening function	.joblib / .py

9. Evaluation rubric (100 marks)

Criteria	Marks	What the teacher looks for
Data audit and cleaning	15	Hidden missing values found, handled correctly, justified
EDA and insights	10	Clear plots, correct observations, not just pictures
Feature engineering	10	At least 3 sensible, justified, leak-free features
Pipelines and validation	15	Stratified split, Pipeline, CV, no leakage
Model comparison and tuning	20	All 6 models, fair comparison, overfit analysis, tuning
Evaluation and threshold tuning	15	Right metrics for imbalance, confusion matrix, recall-focused decision
Interpretation and conclusion	10	Feature importance, limitations, ethics
Code quality, Git, presentation	5	Clean, commented, reproducible, good commit history
Bonus	up to 10	Regression, learning curve, mini app, imputer comparison

10. Common mistakes to avoid

- Reporting **accuracy only**. A model that predicts "healthy" for everyone already gets about 65%.
- Filling missing values on the **whole** dataset before splitting (data leakage).
- Scaling data for tree models and then forgetting to scale for KNN / SVM (or the opposite).
- Choosing the "best" model because it won on the test set. Use cross-validation for that.
- Trusting a Random Forest or XGBoost that scores 100% on training data. Compare it with the validation score!
- Writing "the model is good" without saying *good for whom, at what cost*.

11. Ethics note

This data comes from one specific group of patients. A model trained on it may be unreliable for men, children or other populations, and it is a **screening aid, not a diagnosis**. Always end your report with a short paragraph on limitations and responsible use.

12. Looking ahead

In the next module (**unsupervised learning**) we will return to this same dataset, ignore the Outcome column, and ask a new question: can K-Means or hierarchical clustering discover natural patient groups on its own? Keep your cleaned data and feature-engineering code ready.

Good luck, and remember: a thoughtful, well-explained simple model beats a mysterious complicated one.